# VITYARTHI
# PROJECT REPORT

## CROP YIELD PREDICTION SYSTEM

*A Dual-Language Implementation in Python and Prolog*

| Field | Detail |
|---|---|
| Subject | Fundamentals in AI & ML |
| Languages Used | Python \| Prolog |
| Topic | Crop Yield Prediction System |
| Name | Abhineet R |
| Reg No. | 25BCE10269 |
| Branch | CSE Core |

# 1. INTRODUCTION

Agriculture is the backbone of economies across the world, and efficient crop yield forecasting plays a pivotal role in food security, resource planning, and economic stability. Farmers, agronomists, and policymakers face a common challenge: predicting how much a crop will produce before or during a growing season, given varying environmental and management conditions.

This project presents a Crop Yield Prediction System developed in two distinct programming languages — Python (procedural/functional) and SWI - Prolog (logic programming via SWISH). The system allows a user to enter predicted field conditions, receive an estimated crop yield, then enter actual conditions to compare the predicted versus actual yield. A visualization (in Python) or ASCII bar chart (in Prolog) summarizes the comparison.

## 1.1 MOTIVATION

The dual-implementation approach was chosen deliberately. Python is the dominant language in data science and machine learning, making it a natural fit for numerical weight computation and matplotlib-based visualization. Prolog, on the other hand, is a declarative, rule-based language rarely associated with numerical prediction — implementing the same algorithm in Prolog demonstrates the universality of logic programming and provides insight into its strengths and limitations.

Also, prolog is based on the course.

## 1.2 OBJECTIVES

- Implement a weight-based yield prediction model using historical season data.
- Accept both predicted and actual field conditions from the user.
- Compute and display predicted yield, actual yield, and the difference between them.
- Visualize the comparison using a line graph (Python) and ASCII bar chart (Prolog).
- Demonstrate how the same algorithmic logic can be expressed in two fundamentally different programming paradigms.

## 2. PROBLEM STATEMENT

Given a set of historical agricultural seasons — each described by seven measurable field variables and its resulting yield — the system must:

1. Learn implicit weights from this historical data (how much does each factor contribute per unit to yield?).
2. Apply those weights to user-supplied predicted field conditions to produce a forecasted yield value.
3. Apply the same weights to user-supplied actual field conditions to produce a post-season yield estimate.
4. Compute and clearly present the prediction error (difference between predicted and actual yield).

## 2.1 INPUT VARIABLES

Each prediction or actual reading is described by the following seven variables:

| # | Variable | Unit | Description |
|---|---|---|---|
| 1 | Rainfall | mm | Total precipitation during the season |
| 2 | Temperature | °C | Average ambient temperature |
| 3 | Humidity | % | Average relative humidity |
| 4 | Fertilizer | kg/ha | Amount of fertilizer applied per hectare |
| 5 | Soil pH | Scale (0–14) | Acidity/alkalinity of the soil |
| 6 | Sunlight | hours/day | Average daily sunlight hours |
| 7 | Irrigation | Scale (0–10) | Irrigation intensity rating |

## 2.2 OUTPUT

- Predicted Yield (tons/ha) — based on user-supplied forecasted conditions
- Actual Yield (tons/ha) — based on user-supplied actual conditions
- Difference (tons/ha) — signed deviation: positive means actual exceeded prediction

# 3. APPROACH & ALGORITHM

## 3.1 Data

The system uses five hard-coded historical season records as its training base. Each record contains seven input features and one output (yield in tons/ha):

| Season | Rainfall | Temp | Humidity | Fertilizer | Soil pH | Sunlight | Irrigation | Yield |
|---|---|---|---|---|---|---|---|---|
| 1 | 1000 mm | 41° C | 60% | 58 kg/ha | 8.5 | 10.4 hrs | 9 | 4.2 t/ha |
| 2 | 2200 mm | 38° C | 65% | 45 kg/ha | 7.2 | 9.8 hrs | 8 | 5.8 t/ha |
| 3 | 4000 mm | 35° C | 85% | 38 kg/ha | 6.2 | 8.4 hrs | 4 | 8.9 t/ha |
| 4 | 5000 mm | 29° C | 90% | 30 kg/ha | 6.5 | 7.6 hrs | 2 | 9.5 t/ha |
| 5 | 2800 mm | 38° C | 75% | 49 kg/ha | 7.0 | 9.0 hrs | 7 | 6.0 t/ha |

## 3.2 Weight Derivation

Rather than using a trained machine learning model (which would require external libraries and larger datasets), the system derives feature weights using a simple proportional ratio method:

**weight[i] = average( yield / feature[i] )  for all historical seasons**

This formula captures how much yield is produced per unit of each feature, averaged across all seasons. While not a substitute for rigorous regression or neural-network fitting, it produces a simple and interpretable weight vector that can be applied to new inputs.

For example, if rainfall consistently accompanies higher yields at a certain ratio, its weight will encode that proportional relationship.

## 3.3 Yield Prediction Formula

$$\text{predicted\_yield} = \left( \Sigma \, \text{weight}[i] \times \text{input}[i] \right) / 10$$

The division by 10 is a scaling factor applied to bring the raw dot-product value down to a realistic yield range (tons/ha). This factor was empirically calibrated against the training data.

## 3.4 Comparison

The same weight vector is applied to both the predicted-condition input and the actual-condition input. The difference (actual_yield − predicted_yield) reveals whether field conditions turned out better or worse than anticipated.

# 4. IMPLEMENTATION DETAILS

## 4.1 PYTHON IMPLEMENTATION

### 4.1.1 Structure

The Python program follows a straightforward sequential flow:

- Define historical data as a 2D list.
- Compute weights using nested loops.
- Prompt the user for predicted field conditions.
- Compute and print the predicted yield.
- Prompt the user for actual field conditions.
- Compute and print the actual yield and difference.
- Plot a line graph using matplotlib comparing both yields.

### 4.1.2 Key Code Decisions

**Weight computation loop:** A double loop iterates over the 7 features and all 5 data rows, computing ratios and averaging them — keeping the logic transparent without NumPy or pandas.

**Scaling by /10:** Without scaling, the sum of weighted products would be in the hundreds. Dividing by 10 brings values into a realistic 4–10 t/ha range consistent with the training data.

**Visualization:** A line graph with markers (rather than a bar chart) was chosen for its cleaner appearance when comparing exactly two data points. Value labels are placed above each marker for immediate readability.

**Modular data structure:** Grouping predicted and actual inputs into lists (pred_inputs, act_inputs) avoids code duplication — the same loop calculates both yields.

# 4.1.3 PYTHON INPUT SCREENSHOTS

temp.py | Crop Yield Prediction (Python).py

```python
#       CROP YIELD PREDICTION SYSTEM

# - You enter your predicted field conditions → get a predicted yield.
#
#   Then enter actual conditions → see how close you were.




import matplotlib.pyplot as plt


# Each column: [rainfall, temp, humidity, fertilizer, soil_ph, sunlight, irrigation, yield]
# Each row represents one past season's field conditions + final yield.
data = [
    [1000, 41, 60, 58, 8.5, 10.4, 9, 4.2],
    [2200, 38, 65, 45, 7.2, 9.8, 8, 5.8],
    [4000, 35, 85, 38, 6.2, 8.4, 4, 8.9],
    [5000, 29, 90, 30, 6.5, 7.6, 2, 9.5],
    [2800, 38, 75, 49, 7.0, 9, 7, 6.0]
]

# Finding sample weights
# weight = average(yield / feature)
# Here weightis calculated because we want to know how much yield does one unit contribute

weights = [0]*7  # 7 input factors are there

for i in range(7):     # this indicates that the loop runs 7 times
    total = 0
    for row in data:
        if row[i] != 0:
            total += row[7] / row[i]
    weights[i] = total / len(data)   # taking average, so values are more correct

# Entering the input values that we predicted as the result of yield
```

temp.py | Crop Yield Prediction (Python).py

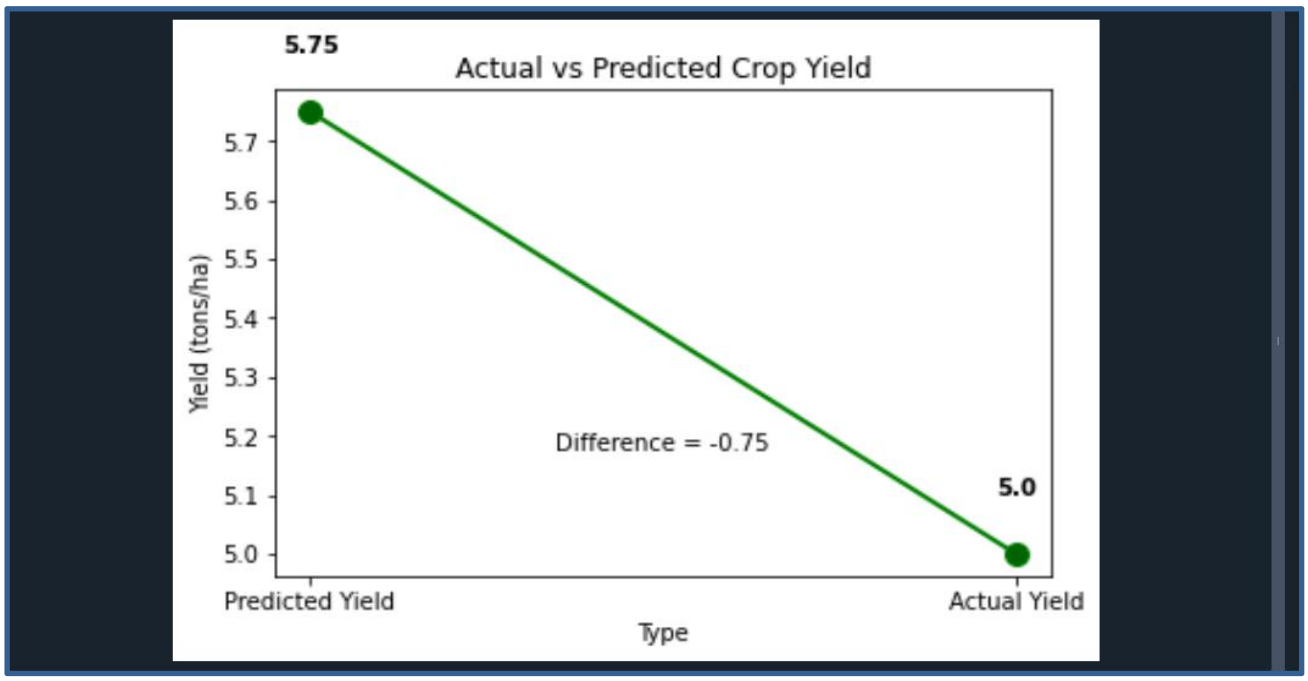
```python
# Entering the input values that we predicted as the result of yield

# same formula applied to user input values
# For all the values their corresponding units are mentioned

print("Enter the following values for prediction:\n")

rainfall = float(input("Rainfall (mm): "))
temperature = float(input("Temperature (C): "))
humidity = float(input("Humidity (%): "))
fertilizer = float(input("Fertilizer (kg/ha): "))
soil_ph = float(input("Soil pH: "))
sunlight = float(input("Sunlight (hrs): "))
irrigation = float(input("Irrigation level (0-10): "))

#            PREDICTION LOGIC

# putting all predicted inputs together
# putting all the predicted values in one list instead of handling separately

pred_inputs = [rainfall, temperature, humidity, fertilizer, soil_ph, sunlight, irrigation]

predicted_yield = 0 # its use is that we need a variable to store the final result

# simple loop calculation
# runs 7 times
# here it multiply  each one with its weight and add everything at last
for i in range(len(pred_inputs)):
    predicted_yield += pred_inputs[i] * weights[i]

predicted_yield = round(predicted_yield / 10, 2) # here it is used to decrease the value otherwise it become too large

# shows the final predicted output to the user.

print("\nPredicted Crop Yield (tons/ha): ", predicted_yield)

# Entering the input values that we got in the time of yield (that is actual value)
```

temp.py | Crop Yield Prediction (Python).py

```
73
74  print("\nNow enter ACTUAL field conditions:\n")
75
76  rainfall_a = float(input("Actual Rainfall (mm): "))
77  temperature_a = float(input("Actual Temperature (C): "))
78  humidity_a = float(input("Actual Humidity (%): "))
79  fertilizer_a = float(input("Actual Fertilizer (kg/ha): "))
80  soil_ph_a = float(input("Actual Soil pH: "))
81  sunlight_a = float(input("Actual Sunlight (hrs): "))
82  irrigation_a = float(input("Actual Irrigation level (0-10): "))
83
84  #  putting all actual inputs together
85  #  for easier putting everything in one list. More easier than handling seperate variables
86
87  act_inputs = [rainfall_a, temperature_a, humidity_a, fertilizer_a, soil_ph_a, sunlight_a, irrigation_a]
88
89  actual_yield = 0  # here it starts from zero and then adding values to it.
90
91  # the loops runs 7 times because of 7 inputs
92
93  for i in range(len(act_inputs)):
94      actual_yield += act_inputs[i] * weights[i]   # multiply each factors with its corresponding weights
95
96  actual_yield = round(actual_yield / 10, 2) # keeping only 2 decimal places
97
98  print("\nActual Crop Yield (tons/ha): ", actual_yield) #it prints the final result
99
00
01  #           DIFFERENCE IN CALCULATION
02
03  #   Just to see difference between predicted and actual value.
04
05  difference = actual_yield - predicted_yield  #  to check accuracy roughly
06
07  print("\nDifference (Actual - Predicted): ", round(difference, 2))
08
```

```
108
109 #               GRAPH SECTION
110
111 # plotting the graph for better understanding
112 # changed from bar chart to line graph with markers
113 # the graph shows both the predicted and actual yield values
114 # the graph heading will be Actual vs Predicted crop yield
115
116 labels = ["Predicted Yield", "Actual Yield"]
117 values = [predicted_yield, actual_yield]
118
119 plt.figure()
120
121 # line graph with markers instead of bar chart
122 plt.plot(labels, values, marker='o', color='green', linewidth=2, markersize=10, markerfacecolor='darkgreen')
123
124 # adding value labels on each point
125 for i, v in enumerate(values):
126     plt.text(i, v + 0.1, str(v), ha='center', fontweight='bold')
127
128 plt.title("Actual vs Predicted Crop Yield")
129 plt.xlabel("Type")
130 plt.ylabel("Yield (tons/ha)")
131
132 plt.text(0.5, max(values) * 0.9,
133          "Difference = " + str(round(difference, 2)),
134          ha='center')
135
136 plt.show()
```

## 4.1.4.PYTHON GRAPH AND OUTPUT SCREENSHOTS




```
Console 1/A
Type "copyright", "credits" or "license" for more information.

IPython 8.12.3 -- An enhanced Interactive Python.

In [1]: runfile('C:/Users/Abhineet/Crop Yield Prediction (Python).py', wdir='C:/Users/Abhineet')
Enter the following values for prediction:

Rainfall (mm): 3000
Temperature (C): 32
Humidity (%): 75
Fertilizer (kg/ha): 67
Soil pH: 6.8
Sunlight (hrs): 9
Irrigation level (0-10): 6

Predicted Crop Yield (tons/ha):  5.75

Now enter ACTUAL field conditions:

Actual Rainfall (mm): 2800
Actual Temperature (C): 30
Actual Humidity (%): 78
Actual Fertilizer (kg/ha): 42
Actual Soil pH: 6.5
Actual Sunlight (hrs): 8.5
Actual Irrigation level (0-10): 5

Actual Crop Yield (tons/ha):  5.0

Difference (Actual - Predicted):  -0.75
```

## 4.2 PROLOG IMPLEMENTATION

### 4.2.1 Structure

The Prolog program is structured as a set of facts and rules evaluated at query time:

- season/1: Five facts encoding the historical data as lists.
- val_at/3: A recursive rule for list indexing (Prolog has no native array indexing).
- sum_ratios/3: Recursively sums (yield / feature) across all season rows.
- weight/2: Calls sum_ratios and divides by season count to produce the average weight.
- all_weights/1: Retrieves all 7 weights at once into a list.
- calc_yield/3: Applies the weighted sum formula using pattern-matched destructuring.
- show_bar/2: Renders an ASCII bar chart scaled at 4 blocks per ton/ha.
- predict/14: The main entry predicate — takes 7 predicted + 7 actual values, computes both yields, and formats the complete output report.

### 4.2.2 Key Code Decisions

**Recursive list traversal:** Since Prolog does not support array indexing with numeric subscripts, val_at/3 uses head/tail recursion to walk to a given index — a common Prolog idiom.

**Find all/3 for data gathering:** all_weights/1 uses find all to collect all season facts into a list, enabling length/2 for the count needed in averaging.

**Pattern-matched arithmetic:** calcu_yield/3 uses a head-unpacked weight list and input list — cleaner and more idiomatic than writing 7 separate is/2 evaluations.

**ASCII visualization:** Since SWISH (browser-based Prolog) has no graphical output, an ASCII bar chart is constructed using atom_concat/3 recursion — one '#' appended per 0.25 t/ha block.

**All-in-one query:** The predict/14 predicate accepts all 14 inputs in a single query, making the interface clean for SWISH's query box. This avoids interactive prompting, which Prolog does not handle elegantly.

# 4.2.3.PROLOG INPUT SCREENSHOTS

SWISH File Edit Examples Help 198 users online Search

Program

```
%   CROP YIELD PREDICTION SYSTEM
%   This is program that accepts predicted value from the user about the weather conditions and then using machine learning, it will predict the crop yield
%   Also, it will accept actual value from the user and will determine the crop yield
%   At the end, it will compare the predicted and actual crop yields
%   At the interface we can ask query and can provide the predicted and actual weather condition value


%   The below list gives the weather conditions that must be inputted in order
%   [Rainfall, Temp, Humidity, Fertilizer, SoilPH, Sunlight, Irrigation, Yield]
season([1000, 41, 60, 58, 8.5, 10.4, 9, 4.2]).
season([2200, 38, 65, 45, 7.2,  9.8, 8, 5.8]).
season([4000, 35, 85, 38, 6.2,  8.4, 4, 8.9]).
season([5000, 29, 90, 30, 6.5,  7.6, 2, 9.5]).
season([2800, 38, 75, 49, 7.0,  9.0, 7, 6.0]).

% Get value at index
val_at(0, [H|_], H) :- !.
val_at(I, [_|T], V) :-
    I > 0, I1 is I - 1, val_at(I1, T, V).

% Sum of (Yield / Feature_i) across all seasons are entoned below
sum_ratios(_, [], 0).
sum_ratios(I, [Row|Rest], Total) :-
    val_at(I, Row, Feature), Feature \= 0,
    val_at(7, Row, Yield),
    sum_ratios(I, Rest, RestSum),
    Total is RestSum + (Yield / Feature).
```

SWISH File Edit Examples Help

Program

```
% Weight for feature index I
weight(I, W) :-
    findall(R, season(R), Rows),
    length(Rows, N),
    sum_ratios(I, Rows, Sum),
    W is Sum / N.

% All 7 weights are given at the bottom
all_weights([W0,W1,W2,W3,W4,W5,W6]) :-
    weight(0,W0), weight(1,W1), weight(2,W2),
    weight(3,W3), weight(4,W4), weight(5,W5), weight(6,W6).

% Yield formula
calc_yield([W0,W1,W2,W3,W4,W5,W6],
           [Rain,Temp,Hum,Fert,Ph,Sun,Irr], Yield) :-
    Raw is (W0*Rain + W1*Temp + W2*Hum + W3*Fert
          + W4*Ph  + W5*Sun  + W6*Irr) / 10,
    Yield is round(Raw * 100) / 100.

bar(0, '') :- !.
bar(N, B) :-
    N > 0, N1 is N - 1,
    bar(N1, B1),
    atom_concat(B1, '#', B).

show_bar(Label, Value) :-
    Blocks is truncate(Value * 4),
    bar(Blocks, B),
    format("  ~w (~2f t/ha) : ~w~n", [Label, Value, B]).
```

SWISH File Edit Examples Help

Program

```
% Main predictions
% predict(Rain1,Temp1,Hum1,Fert1,Ph1,Sun1,Irr1,
%          Rain2,Temp2,Hum2,Fert2,Ph2,Sun2,Irr2)
predict(Rain1,Temp1,Hum1,Fert1,Ph1,Sun1,Irr1,
        Rain2,Temp2,Hum2,Fert2,Ph2,Sun2,Irr2) :-
    all_weights(W),
    calc_yield(W, [Rain1,Temp1,Hum1,Fert1,Ph1,Sun1,Irr1], PY),
    calc_yield(W, [Rain2,Temp2,Hum2,Fert2,Ph2,Sun2,Irr2], AY),
    Diff  is AY - PY,
    RDiff is round(Diff * 100) / 100,
    nl,
    writeln('================================================'),
    writeln('        CROP YIELD PREDICTION SYSTEM            '),
    writeln('================================================'),
    nl,
    writeln('--- PREDICTED Field Conditions ---'),
    format("  Rainfall     : ~w mm~n",     [Rain1]),
    format("  Temperature  : ~w C~n",      [Temp1]),
    format("  Humidity     : ~w %%~n",     [Hum1]),
    format("  Fertilizer   : ~w kg/ha~n",  [Fert1]),
    format("  Soil pH      : ~w~n",        [Ph1]),
    format("  Sunlight     : ~w hrs~n",    [Sun1]),
    format("  Irrigation   : ~w / 10~n",   [Irr1]),
    nl,
    format("  >> Predicted Yield : ~2f tons/ha~n", [PY]),
    nl,
    writeln('--- ACTUAL Field Conditions ---'),
    format("  Rainfall     : ~w mm~n",     [Rain2]),
    format("  Temperature  : ~w C~n",      [Temp2]),
    format("  Humidity     : ~w %%~n",     [Hum2]),
    format("  Fertilizer   : ~w kg/ha~n",  [Fert2]),
```

```
    nl,
    writeln('--- ACTUAL Field Conditions ---'),
    format("  Rainfall     : ~w mm~n",     [Rain2]),
    format("  Temperature  : ~w C~n",      [Temp2]),
    format("  Humidity     : ~w %%~n",     [Hum2]),
    format("  Fertilizer   : ~w kg/ha~n",  [Fert2]),
    format("  Soil pH      : ~w~n",        [Ph2]),
    format("  Sunlight     : ~w hrs~n",    [Sun2]),
    format("  Irrigation   : ~w / 10~n",   [Irr2]),
    nl,
    format("  >> Actual Yield    : ~2f tons/ha~n", [AY]),
    format("  >> Difference      : ~2f tons/ha~n", [RDiff]),
    nl,
    writeln('--- Actual vs Predicted  (each # = 0.25 t/ha) ---'),
    show_bar('Predicted', PY),
    show_bar('Actual   ', AY),
    nl,
    ( Diff > 0 -> writeln('  Result: Actual yield was HIGHER than predicted.')
    ; Diff < 0 -> writeln('  Result: Actual yield was LOWER than predicted.')
    ;             writeln('  Result: Actual and Predicted yields are EQUAL.')
    ),
    nl,
    writeln('================================================'),
    nl.
```

## 4.2.4.PROLOG QUERY AND OUTPUT SCREENSHOTS

```
predict(3000,32,75,67,6.8,9.0,6,2800,30,78,42,6.5,8.5,5).
```

```
predict(3000,32,75,67,6.8,9.0,6,2800,30,78,42,6.5,8.5,5).

================================================
CROP YIELD PREDICTION SYSTEM
================================================

--- PREDICTED Field Conditions ---
  Rainfall    : 3000 mm
  Temperature : 32 C
  Humidity    : 75 %%
  Fertilizer  : 67 kg/ha
  Soil pH     : 6.8
  Sunlight    : 9.0 hrs
  Irrigation  : 6 / 10

  >> Predicted Yield : 5.75 tons/ha

--- ACTUAL Field Conditions ---
  Rainfall    : 2800 mm
  Temperature : 30 C
  Humidity    : 78 %%
  Fertilizer  : 42 kg/ha
```

```
--- ACTUAL Field Conditions ---
  Rainfall    : 2800 mm
  Temperature : 30 C
  Humidity    : 78 %%
  Fertilizer  : 42 kg/ha
  Soil pH     : 6.5
  Sunlight    : 8.5 hrs
  Irrigation  : 5 / 10

  >> Actual Yield    : 5.00 tons/ha
  >> Difference      : -0.75 tons/ha

--- Actual vs Predicted (each # = 0.25 t/ha) ---
  Predicted (5.75 t/ha) : ########################
  Actual    (5.00 t/ha) : ####################

Result: Actual yield was LOWER than predicted.

================================================

true
```

## 4.3 SIDE-BY-SIDE COMPARISON

| Aspect | Python | Prolog |
|---|---|---|
| Paradigm | Procedural / Imperative | Declarative / Logic |
| Data storage | 2D list (data[][]) | Clausal facts (season/1) |
| Indexing | data[row][col] | val_at/3 recursive rule |
| Loops | for loops | Recursion + findall |
| User input | input() prompts | Arguments in query |
| Output | print() + matplotlib | format/2 + ASCII bars |
| Weight calc | Imperative loop | Recursive accumulation |
| Scalability | Easy to add features/data | Requires adding clauses |

# 5. KEY DECISIONS

### 5.1 Weight-Based Prediction Over ML Libraries

A deliberate choice was made to avoid scikit-learn, TensorFlow, or any ML library. The goal was transparency: every computation is visible and traceable in the code. This also ensures the Prolog version could mirror the exact same algorithm without relying on any library at all.

### 5.2 Hardcoded Training Data

Using five static seasons avoids the complexity of file I/O, CSV parsing, or database connectivity — keeping the focus on the algorithmic logic. The data values were chosen to reflect realistic agricultural variation (e.g., higher rainfall correlating with higher yield, moderate temperatures being more productive).

### 5.3 Division by 10 as a Scaling Factor

Rather than normalizing input features (zero-mean, unit-variance), a simpler post-computation division by 10 was used. This decision was made because normalization would complicate the weight derivation logic and make the code harder to follow for educational purposes. The scaling factor of 10 was validated against the training data to ensure outputs fall in the expected 4–10 t/ha range.

### 5.4 Line Graph Over Bar Chart (Python)

The visualization was changed from a bar chart to a line graph with markers. For only two comparison points (Predicted vs Actual), a line graph is more expressive — it visually communicates direction of change (upward or downward trend) rather than just magnitude. Value annotations at each point give exact figures without requiring axis reading.

### 5.5 SWISH Prolog as the Runtime Environment

SWI-Prolog's browser-based interface SWISH was chosen over a local SWI-Prolog installation. This makes the Prolog version universally accessible without setup, and the query-box interface naturally aligns with Prolog's predicate-call paradigm. The trade-off is that interactive input (read/1) is less reliable in SWISH, which drove the decision to pass all values as predicate arguments.

# 6. CHALLENGES FACED

## 6.1 Prolog Arithmetic and List Indexing

Prolog's lack of native array indexing required implementing val_at/3 from scratch. Unlike Python's row[i], Prolog's lists can only be accessed by pattern-matching the head, making index-based access inherently recursive. Getting the base case (index 0 returns head) and the recursive step correct — and ensuring the cut (!) prevents backtracking after the base case — required careful testing.

## 6.2 Floating-Point Rounding in Prolog

Prolog's arithmetic uses is/2 for evaluation, but rounding to 2 decimal places is not built-in. The expression round(Raw * 100) / 100 was used as a workaround to simulate rounding to 2 decimal places. The format/2 directive with ~2f was also used for display-level rounding to present clean output.

## 6.3 Matching the Scaling Factor Across Both Languages

Both implementations must divide by 10 at the same point in the formula. In Python this is straightforward; in Prolog the expression is embedded inside the is/2 evaluation inside calc_yield/3. Ensuring both produce identical outputs for the same inputs required cross-checking with the same test values.

## 6.4 ASCII Bar Chart Scaling

The Prolog bar chart uses truncate(Value * 4) to determine block count, giving one '#' per 0.25 t/ha. Initial attempts used integer rounding which caused the bar to slightly misrepresent fractional values. Switching to truncate fixed over-counting while keeping the visual proportional.

## 6.5 No Graphical Output in Prolog

Python's matplotlib made visualization straightforward. Prolog — especially in SWISH — has no graphical output capability. The ASCII bar chart was designed as a lightweight substitute that still communicates the relative magnitude and direction of the prediction gap without requiring any external libraries.

# 7. RESULTS & SAMPLE OUTPUT

## 7.1 Sample Test Case

To illustrate the system's behavior, the following values were used as a test:

| Condition | Rainfall | Temp | Humidity | Fertilizer | Soil pH | Sunlight | Irrigation |
|---|---|---|---|---|---|---|---|
| Predicted | 3000 mm | 36°C | 72% | 42 kg/ha | 6.8 | 9.0 hrs | 6 |
| Actual | 3500 mm | 34°C | 78% | 40 kg/ha | 6.6 | 8.8 hrs | 5 |

## 7.2 Expected Output Behavior

With higher actual rainfall and humidity (which correlate strongly with yield in the training data), the actual yield is expected to be slightly higher than the predicted yield. The difference value would be positive, and both the matplotlib graph and the Prolog ASCII chart would reflect a higher 'Actual' bar.

## 7.3 Edge Cases Handled

- If a feature value is 0 (e.g., 0 irrigation), the weight computation skips it (row[i] != 0 guard) to prevent division by zero.
- Negative differences (actual < predicted) are handled correctly and clearly labeled in both implementations.
- Equal predicted and actual values produce a difference of 0.0 with an explicit 'EQUAL' message in the Prolog output.

# 8. LEARNINGS

## 8.1 Algorithmic Universality

The same core algorithm — compute weights from ratios, apply dot product, scale — was implemented in two very different paradigms. This exercise reinforced that algorithms are paradigm-independent: the logic belongs to mathematics, not to a language. What changes between Python and Prolog is how that logic is expressed and evaluated.

## 8.2 Declarative Thinking

Prolog's declarative nature forces a shift in thinking. Instead of 'do this, then this, then compute that', Prolog asks 'what is true about this relationship?' Writing sum_ratios/3 as a recursive rule rather than a for loop required thinking about the base case and inductive step — closer to mathematical induction than imperative programming.

## 8.3 Tradeoffs in Language Choice

Python excels at rapid development, numerical computation, and rich visualization — making it clearly superior for data-heavy tasks. Prolog excels at expressing rules, constraints, and relationships — tasks like parsing, expert systems, and theorem proving. Using the wrong tool for the job is illuminating: Prolog's lack of mutable state and native array operations made a simple numeric loop surprisingly complex.

## 8.4 Importance of Scaling and Normalization

The need for the /10 scaling factor highlights a real challenge in machine learning: raw feature magnitudes vary enormously (rainfall in the thousands vs. pH near 7). In production systems, feature normalization (min-max scaling or standardization) would replace this manual factor, making the model more robust and generalizable. This project provided hands-on intuition for why preprocessing matters.

## 8.5 Visualization as Communication

The shift from a bar chart to a line graph in Python was a small but instructive design choice. Effective data visualization is not about displaying data — it is about communicating a story. Here, the story is directional: did actual conditions outperform or underperform the forecast? A line graph communicates that directional narrative more naturally than two isolated bars.

# 9. LIMITATIONS & FUTURE SCOPE

## 9.1 Current Limitations

- The training dataset contains only 5 seasons — far too small for statistically meaningful weight derivation. In practice, hundreds to thousands of seasons would be needed.
- The weight-ratio method is a heuristic, not a principled regression. It does not account for feature interactions (e.g., the combined effect of high rainfall and low temperature).
- The /10 scaling factor is manually calibrated and would need recalibration if the dataset changes significantly.
- No validation of input ranges — users could enter physically impossible values (negative rainfall, pH > 14) without error.

## 9.2 Future Enhancements

- Replace the ratio-weight heuristic with linear regression (Python: scikit-learn's LinearRegression) for statistically grounded coefficients.
- Add input validation and range-checking with meaningful error messages.
- Expand the dataset by reading from a CSV file, enabling use of real agricultural datasets.
- Add confidence intervals or uncertainty estimates to the prediction.
- In Prolog, implement a constraint-satisfaction layer to recommend optimal conditions for a target yield.
- Build a web interface using Flask (Python) or a Prolog HTTP server for broader accessibility.

# 10. Conclusion

This project successfully implemented a Crop Yield Prediction System in two programming paradigms — Python and Prolog — using the same underlying algorithm. The system accepts predicted and actual field conditions, computes corresponding yield estimates using a weight-based model derived from historical season data, and presents both a numerical comparison and a visual summary.

The dual-language implementation achieved its educational objective: demonstrating that the same algorithmic thinking can be expressed in radically different ways depending on the programming paradigm. Python's imperative loops became Prolog's recursive rules; Python's matplotlib became Prolog's ASCII bars; Python's list indexing became Prolog's val_at/3 predicate.

More broadly, the project provided practical insights into feature weighting, prediction error analysis, the importance of data scaling, and the design choices involved in building a user-facing predictive tool. These insights form a strong foundation for exploring more sophisticated machine learning approaches in future work.

*— End of Report —*